
✅1. Object-Oriented Programming (OOP)

➤ What is OOP?

Object-Oriented Programming (OOP) is a paradigm that organizes software design around data, or objects, rather than functions and logic. An object is an instance of a class, and classes define the blueprint of objects.

➤ Key Concepts:

1. **Encapsulation**: Hides internal state and only exposes operations through methods. This improves modularity and prevents external code from changing internal object details directly.

```
class BankAccount {
    private:
        int balance;
    public:
        void deposit(int amount) { balance += amount; }
};
```

1. **Abstraction**: Focuses on what an object does instead of how it does it. It hides unnecessary details from the user.

2. **Inheritance**: Allows a class (child) to inherit attributes and methods from another class (parent). This helps code reuse.

```
class Animal {
public:
    void eat() { cout << "Eating"; }
};
class Dog : public Animal {
    void bark() { cout << "Barking"; }
};
```

1. **Polymorphism**: Allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation.

```
class Animal {
    public: virtual void makeSound() { cout << "Some sound"; }
};
```

```cpp
class Cat : public Animal {
    void makeSound() override { cout << "Meow"; }
};
```

✅2. Pointers and Raw Pointers

➤ What are Pointers?

Pointers are variables that store memory addresses. They allow direct memory access and manipulation, which can be powerful but dangerous.

➤ Why Use Them?

- Dynamic memory allocation
- Passing large objects efficiently
- Building complex data structures (linked list, trees)

➤ Syntax:

```cpp
int x = 5;
int* ptr = &x;  // ptr holds the address of x
cout << *ptr;   // dereferencing the pointer prints value of x
```

➤ Issues with Raw Pointers:

- Manual memory management (`new` / `delete`)
- Memory leaks
- Dangling pointers

✅3. Smart Pointers

➤ What are Smart Pointers?

Smart pointers are class templates in C++ that provide automatic memory management. They automatically release memory when the object is no longer in use.

➤ Types:

1. `unique_ptr` – Exclusive ownership. Can't be copied.
2. `shared_ptr` – Reference counting. Multiple shared owners.
3. `weak_ptr` – Non-owning reference. Used to prevent cyclic dependencies.

➤ Example:

```
shared_ptr<int> sp1 = make_shared<int>(10);
shared_ptr<int> sp2 = sp1;  // both share ownership
cout << sp1.use_count();    // prints 2
```

✅4. Threads and Multithreading

➤ What is Multithreading?

Multithreading allows concurrent execution of two or more threads. Each thread represents a separate path of execution.

➤ Key Concepts:

- **thread creation**: `std::thread`
- **joining**: waits for thread to finish
- **detaching**: separates thread from main
- **mutex**: ensures mutual exclusion to avoid race conditions
- **condition_variable**: used for thread synchronization

➤ Example:

```
void task() { cout << "Running"; }
thread t(task);
t.join();
```

✅5. File Handling

➤ What is File I/O?

C++ uses `fstream` to perform input and output operations on files. Modes include:

- `ios::in` – read
- `ios::out` – write
- `ios::app` – append
- `ios::trunc` – truncate file

➤ Reading & Writing:

```
fstream file("data.txt", ios::out);
file << "Hello world";
file.close();

file.open("data.txt", ios::in);
string line;
getline(file, line);
cout << line;
```

➤ Important Methods:

- `seekg()`, `seekp()`: change get/put pointer
- `tellg()`, `tellp()`: get current position
- `clear()`: clear error flags

✅6. Exception Handling

➤ Why Exceptions?

To handle runtime errors gracefully without crashing the program.

➤ Syntax:

```
try {
    if (somethingWrong)
        throw runtime_error("Problem");
} catch (const exception &e) {
    cout << e.what();
}
```

➤ Custom Exception:

```
class MyError : public exception {
    const char* what() const noexcept override {
        return "Custom error occurred!";
    }
};
```

✅7. Vtable and Vptr

➤ What are They?

- `vtable`: Virtual table. A per-class table holding function pointers for virtual functions.
- `vptr`: Virtual pointer. A hidden pointer inside objects pointing to the class's vtable.

➤ Used For:

- Resolving function calls at runtime (runtime polymorphism)

➤ Example:

```
class A {
    public: virtual void show() { cout << "A"; }
};
class B : public A {
    void show() override { cout << "B"; }
};
A* ptr = new B();
ptr->show();  // Calls B::show via vptr and vtable
```

✅8. Templates

➤ Why Templates?

To write generic code that works for any data type.

➤ Function Template:

```
template<typename T>
T add(T a, T b) {
    return a + b;
}
```

➤ Class Template:

```
template<typename T>
class Box {
    T data;
};
```

✅9. STL (Standard Template Library)

➤ Components:

- **Containers**: Hold data (`vector`, `stack`, `queue`, `set`, `map`)
- **Algorithms**: Work on containers (`sort`, `find`, `binary_search`)
- **Iterators**: Act like pointers for containers

➤ Example:

```
vector<int> v = {4, 1, 3};
sort(v.begin(), v.end());
for (int x : v) cout << x;
```

✅10. C++20 Basics

➤ Major Features:

1. **Concepts**: Constraints on template types.

```

```

template concept Addable = requires(T a, T b) { a + b; };

```
2. **Ranges**: Cleaner iteration and views.
   ```cpp
#include <ranges>
for (int i : std::views::iota(1, 5)) cout << i;
```

1. `** and **`: Compile-time evaluation
2. **Coroutines**: Async programming style using `co_await`, `co_yield`

🔗 Use this guide to revise and explain each topic confidently in your interviews. Shall we export this as a downloadable PDF now?
```